Project Charter — Smart Traffic Light Controller with Adaptive Flow

1. Project Domain

This project belongs to the domain of *intelligent transportation systems* and *traffic simulation*. It models real-world intersections and applies adaptive signal-control algorithms to reduce congestion and optimize vehicle flow.

2. Client Scenario

The City Traffic Control Department faces severe congestion during peak hours because current traffic lights operate on fixed timers. These timers do not respond to real-time traffic variations, causing long queues, increased waiting time, and inefficient green-time allocation.

The department needs a **software-based simulator** that models multiple intersections, measures varying traffic load, and dynamically adapts signal timings using intelligent algorithms. The simulator will rely on virtual sensor inputs (simulated vehicle arrivals), not physical hardware.

3. Project Goals

1. Reduce average waiting time by prioritizing the most congested lanes
2. Dynamically adjust green-light duration based on queue length
3. Model multi-intersection networks using graphs
4. Collect performance metrics for analysis
5. Demonstrate a realistic, console-based traffic simulation

4. System Modules

Module	Purpose
Simulator	Handles time-steps, car arrivals, and controls the main program loop.
Controller	Implements the adaptive algorithm: selects the best lane and sets green times.
Lane Manager	Maintains queues for N/S/E/W lanes and updates queue statistics.
Graph Manager	Represents intersections and their connections; routes vehicles between nodes.
Metrics Manager	Computes wait time, throughput, max queue length, and cycle statistics.
Visualizer	Console-based display of queues, signal states, and cycle outputs.

---

5. Data Structures and Justification

1. Queue (Lane Storage)

1. Models real-life FIFO vehicle flow
2. O(1) enqueue/dequeue
3. Alternatives like stacks or lists break ordering → invalid

2. Graph (Adjacency List for Intersections)

1. City roads form a sparse network → adjacency list is memory-efficient
2. Matrix wastes memory and is harder to manage
3. Enables multi-intersection routing

3. HashMap (Per-Intersection Stats)

1. Stores dynamic statistics: avg wait, queue lengths, cycles
2. Allows O(1) access with flexible intersection IDs
3. Arrays require numeric IDs, unsuitable

4. Heap / Priority Queue (Lane Selection)

1. Efficiently picks lane with highest priority (queue length)
2. O(log n) insertion, O(1) top
3. Sorting each cycle is inefficient

5. Arrays (Cycle Logs & Metrics)

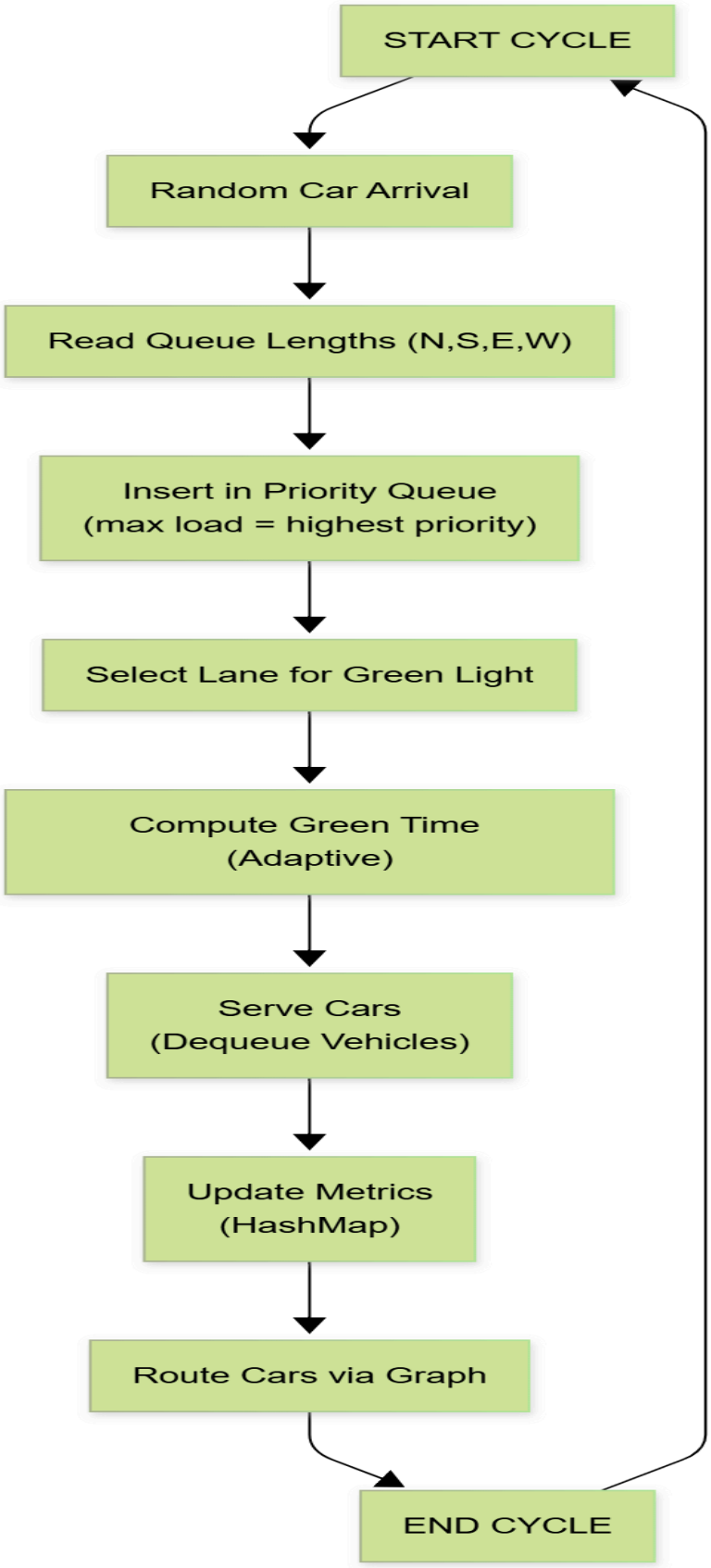
1. Sequential time steps → perfect for ordered arrays
2. Fast indexing and iteration
3. Ideal for throughput/time graphs later

6. Algorithm Overview

1. Generate vehicle arrivals probabilistically
2. Read queue lengths for all lanes
3. Insert lane priorities into a max-heap
4. Select lane with highest load
5. Compute green-time using adaptive formula
6. Dequeue vehicles accordingly
7. Update stats in HashMap
8. Move vehicles across graph edges
9. Store metrics in arrays
10. Repeat next cycle

7. Basic Simulation Scenario

1. 10 cars arrive on North → controller gives longer green
2. 0 cars on East → lane skipped
3. West has medium load → medium green
4. Stats update; next cycle begins
5. Cars move to connected intersections using the graph


START CYCLE
Random Car Arrival
Read Queue Lengths (N,S,E,W)
Insert in Priority Queue
(max load = highest priority)
Select Lane for Green Light
Compute Green Time
(Adaptive)
Serve Cars
(Dequeue Vehicles)
Update Metrics
(HashMap)
Route Cars via Graph
END CYCLE

8. Constraints

1. Console-based simulation
2. C++ only
3. Efficient data structures required
4. No GUI framework required